

Kubernetes & Container Orchestration — Practical System Design Notes

Hinglish • Docker • Pods • Deployments • Services • Ingress • HPA • Health Checks • Production

1. Container Orchestration Kya Hai?

Jab aapke paas dozens/hundreds of containers hon, manually start/stop/restart/scale/network karna difficult ho jata hai. Container orchestration platform ye operational work automate karta hai.

100 containers

Need:

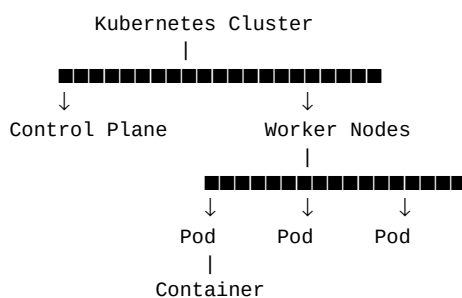
- scheduling
- restart
- scaling
- networking
- health checks
- deployment
- service discovery

Kubernetes = container orchestration platform

2. Docker vs Kubernetes

Docker	Kubernetes
Build/run containers	Manage containers at cluster scale
Container image/runtime workflow	Scheduling + networking + scaling
Single machine workflows easy	Multi-node orchestration
docker run	Deployment/Pod/Service etc.

3. Kubernetes Mental Model



4. Cluster

Kubernetes cluster machines ka group hai jo workloads run karta hai. Broadly control plane cluster ko manage karta hai aur worker nodes application workloads run karte hain.

5. Control Plane

- API Server — Kubernetes API ka entry point.
- Scheduler — Pod ko suitable node par place karta hai.
- Controller Manager — desired state ko actual state ke saath reconcile karta hai.
- etcd — cluster state/configuration ka distributed key-value store.

6. Worker Node

- Kubelet — node par Pods/containers ki lifecycle manage karta hai.
- Container runtime — containers execute karta hai.
- Networking components — Pod/service networking support.

7. Desired State

You declare:

```
replicas = 3
```

Kubernetes observes:
only 2 running

Controller:
→ creates another Pod

Actual state → desired state

Kubernetes ka core mental model declarative hai: aap desired state define karte ho, controllers continuously us state ko maintain karte hain.

8. Pod

Pod Kubernetes ka smallest deployable unit hai. Usually ek Pod mein ek main application container hota hai, but tightly coupled sidecars bhi same Pod mein ho sakte hain.

Pod
■■■ Node.js container
■■■ optional sidecar

Containers same Pod mein network namespace share kar sakte hain aur localhost ke through communicate kar sakte hain.

9. Pod ≠ Permanent Server

Pod disposable/replaceable hota hai. Pod delete/crash ho sakta hai aur controller new Pod create kar sakta hai. Isliye application state ko Pod local filesystem/memory mein critical persistent state ke liye rely nahi karna chahiye.

10. Deployment

Deployment stateless application Pods ke desired replica count aur rollout ko manage karne ka common abstraction hai.

Deployment
↓
ReplicaSet
↓
Pod Pod Pod
↓
Node.js containers

11. Replica

replicas: 3

Pod A
Pod B
Pod C

Agar ek Pod fail ho jaye aur Deployment desired replicas 3 rakhta hai, controller replacement create karne ki koshish karega.

12. Service

Pod IPs dynamic ho sakte hain. Kubernetes Service stable network endpoint provide karti hai.

Client
↓
user-service
↓
Pod A
Pod B
Pod C

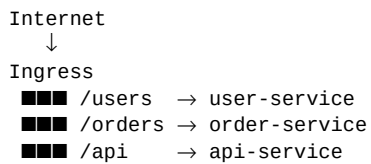
Service traffic ko matching healthy/available Pods tak route karne ka stable abstraction provide karti hai.

13. Service Types

- ClusterIP — cluster-internal default service.
- NodePort — node-level port expose karta hai.
- LoadBalancer — cloud/platform load balancer integration ke liye.

14. Ingress

Ingress HTTP/HTTPS routing rules define kar sakta hai, often host/path based.



Modern Kubernetes environments mein Gateway API bhi HTTP routing ke liye relevant standard hai; Ingress remains widely used.

15. Ingress vs API Gateway

Ingress	API Gateway
Kubernetes HTTP routing abstraction	Broader API traffic management
Path/host routing	Auth/rate limits/policies etc.
Often paired with ingress controller	Can sit at broader API boundary

16. ConfigMap

Non-secret configuration ko Pod/application se separate rakhne ke liye ConfigMap use ho sakta hai.

```
ConfigMap:  
PORT=3000  
LOG_LEVEL=info
```

```
Pod  
↓  
environment/config file
```

17. Secret

Passwords, API keys and other sensitive configuration ke liye Kubernetes Secret abstraction use hota hai. Secret ko automatically 'encrypted everywhere' assume mat karo; cluster configuration, RBAC and secret-at-rest protection matter karte hain.

```
Secret  
↓  
Pod  
↓  
DB_PASSWORD
```

18. Liveness Probe

Liveness probe ka purpose determine karna hai ki container/process ko restart ki zaroorat hai ya nahi.

```
Liveness:  
GET /health/live  
  
fails repeatedly  
↓  
container restart
```

19. Readiness Probe

Readiness batati hai ki Pod traffic receive karne ke liye ready hai ya nahi.

```
Readiness:  
DB unavailable  
↓  
Pod NOT READY  
↓  
Service traffic avoid
```

Ye distinction bahut important hai: unhealthy dependency ke time app ko traffic se temporarily remove karna restart karne se better ho sakta hai.

20. Startup Probe

Slow-starting applications ke liye startup probe startup phase ko protect kar sakta hai, so liveness checks app fully initialized hone se pehle restart na karein.

21. Health Endpoint

```
GET /health/live  
→ process is alive
```

```
GET /health/ready  
→ dependencies/config allow traffic
```

Health checks ko overly deep mat banao. Agar readiness endpoint har optional dependency ko check karega, temporary issue par unnecessarily traffic remove ho sakta hai.

22. Resource Requests

CPU/memory request scheduler ko batata hai ki workload ko approximately kitne resources reserve/schedule karne chahiye.

```
resources:
  requests:
    cpu: 250m
    memory: 256Mi
```

23. Resource Limits

Limits container ke resource consumption par upper boundary define karte hain. CPU and memory limits ke behavior different ho sakte hain; memory pressure/OOM ko understand karna important hai.

```
resources:
  limits:
    cpu: 1
    memory: 512Mi
```

24. Why Requests Matter

Agar requests realistic nahi hain, scheduler poor placement kar sakta hai ya cluster capacity planning inaccurate ho sakti hai.

25. Horizontal Pod Autoscaler — HPA

HPA workload metrics ke basis par Pod replicas automatically adjust kar sakta hai.

```
Traffic ↑  
↓  
CPU / custom metric ↑  
↓  
HPA  
↓  
replicas:  
3 → 6
```

HPA tab useful hai jab application horizontally scalable ho aur required metrics available hon.

26. HPA Example

```
minReplicas: 2  
maxReplicas: 10  
target CPU: 60%
```

```
CPU > target  
→ scale out
```

```
CPU < target  
→ scale in
```

27. HPA Is Not Magic

CPU low hone ke bawajood application DB, external API ya queue bottleneck se slow ho sakti hai. CPU-only autoscaling har workload ke liye sufficient nahi.

28. Deployment Strategies

- Rolling update — gradually old Pods replace.
- Blue/green — separate environments and switch traffic.
- Canary — small percentage traffic new version ko.

```
v1 → 90%  
v2 → 10%
```

```
Observe:  
errors  
latency  
business metrics
```

Then increase v2

29. Rolling Deployment

```
Old:  
Pod v1  
Pod v1  
Pod v1
```

```
Rollout:
```

```
Pod v1  
Pod v1  
Pod v2
```

```
Then:
```

```
Pod v1  
Pod v2  
Pod v2
```

```
Finally:
```

Pod v2
Pod v2
Pod v2

30. Rollback

Bad deployment ke baad previous known-good revision par rollback important operational capability hai. Deployment history aur health checks release safety improve karte hain.

31. Namespace

Namespace cluster resources ko logical groups mein organize karne ka mechanism hai.

```
cluster
  ■■■ dev
  ■■■ staging
  ■■■ production
```

Namespace isolation useful hai, but namespace itself complete security boundary nahi hai; RBAC/network policies etc. bhi matter karte hain.

32. Labels & Selectors

```
Pod:
app=payment
version=v2
```

```
Service selector:
app=payment
```

```
Service
↓
matching Pods
```

Labels Kubernetes resources ko identify/group karne ka core mechanism hain.

33. Service Discovery

Order Pod
↓
payment-service
↓
Kubernetes DNS
↓
Payment Service
↓
Payment Pods

Kubernetes Service DNS stable service-to-service discovery provide kar sakta hai.

34. Persistent Storage

Pods disposable hote hain, so databases/file state ke liye persistent storage abstraction chahiye. Kubernetes PersistentVolume (PV) and PersistentVolumeClaim (PVC) commonly used concepts hain.

Pod
↓
PVC
↓
PV
↓
Persistent storage

35. Stateful vs Stateless

Stateless	Stateful
Session/data externalized	Identity/storage state important
Easy horizontal scaling	Requires stable identity/storage patterns
Typical Node API	Database/ordered cluster workloads

36. StatefulSet

StatefulSet stateful workloads ko stable identity and storage patterns ke saath manage karne ke liye designed abstraction hai. Databases ko blindly StatefulSet mein run karna production best practice nahi; managed database service often simpler.

37. DaemonSet

DemonSet ka goal typically har eligible node par Pod ka instance run karna hai. Logging agents, monitoring agents jaise workloads common examples hain.

38. Job

Job finite task ko successfully complete karne ke liye useful hai.

Job
↓
migration script
↓
complete

39. CronJob

CronJob scheduled Jobs run kar sakta hai.

CronJob
↓
daily 02:00
↓

backup/report/cleanup Job

40. Node Failure

Pod A



Node 1 crashes

Kubernetes:

Pod replacement



Node 2

Actual behavior workload controller, scheduling constraints, storage and cluster health par depend karta hai.

41. Pod Affinity / Anti-Affinity

Scheduling rules se related Pods ko near ya apart place karne ki preference/requirement express ki ja sakti hai. High availability ke liye replicas ko same failure domain mein concentrate na karna useful ho sakta hai.

42. Taints & Tolerations

Taints node ko certain Pods se protect kar sakte hain; tolerations matching Pods ko tainted node par schedule hone allow karte hain. Specialized nodes ke liye useful.

43. Network Policy

NetworkPolicy Pod-to-Pod traffic ko restrict/control karne ke liye use ho sakti hai, provided cluster networking implementation supports it.

Default:
Pod A → Pod B

Policy:
only
Order → Payment
allow

Others:
deny

44. RBAC

Role-Based Access Control define karta hai kaun Kubernetes resources par kya actions perform kar sakta hai.

Developer:
get/list pods

CI/CD:
deploy/update specific resources

Admin:
broader permissions

Least privilege important hai.

45. Horizontal vs Vertical Scaling

Horizontal	Vertical
More Pods/instances	More CPU/RAM per instance
Good for stateless APIs	May have hardware/size limits
Kubernetes HPA common	VPA can adjust resource recommendations/requests

46. Kubernetes + Redis + DB

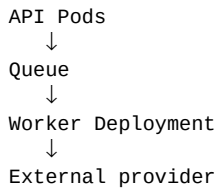
Ingress
↓
Service
↓
Node.js Pods × N
■ → Redis
■ → DB

HPA:
Node Pods 2 → 8

Important:
DB capacity may become bottleneck
even when Pods scale.

Application scaling automatically database scaling solve nahi karta. End-to-end capacity dependency chain ke bottleneck par constrained hoti hai.

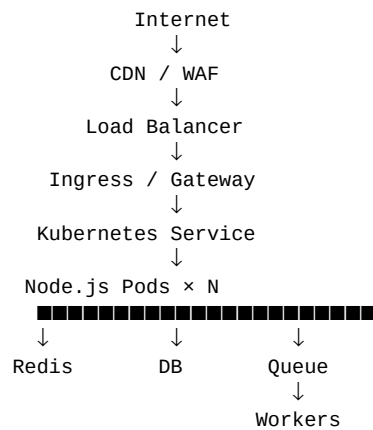
47. Kubernetes + Queue



API scale and worker scale independently.

Worker count ko queue depth/lag aur downstream limits ke according scale karna useful hai.

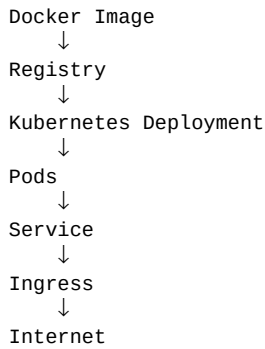
48. Production Architecture



Observability:

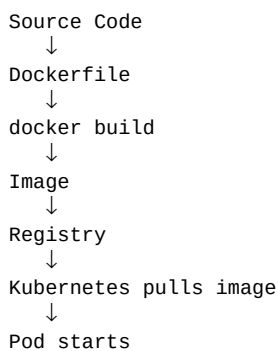
Prometheus + Grafana + Logs + Traces

49. Node.js Deployment Mental Model



Node.js app ko container image mein package karke registry mein push karo, then Kubernetes Deployment desired replicas and rollout manage kare.

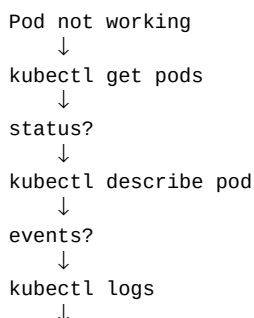
50. Minimal Docker Flow



51. Kubernetes Failure Scenarios

Problem	Handling
Pod crashes	Controller creates replacement if managed by Deployment/etc.
Readiness fails	Pod removed from Service traffic
Liveness fails	Container may be restarted
Node fails	Scheduler/controller may replace workload elsewhere if capacity allows
Bad deployment	Rollback / rollout pause
CPU saturation	HPA/optimization/resource tuning
Memory OOM	Inspect limits, heap, leaks, workload
DB unavailable	App readiness/failure handling; don't blindly restart everything
Image pull failure	Check registry/auth/image/tag/network

52. Kubernetes Debugging Flow



application error?
↓
Service/endpoints?
↓
Ingress?
↓
DNS/network?
↓
resource limits?
↓
dependency DB/Redis?

53. Common Statuses

- Pending — Pod scheduling/image/resource issue may exist.
- Running — containers are running, but app may still be unready.
- Succeeded — finite workload completed.
- Failed — finite workload failed.
- CrashLoopBackOff — container repeatedly crashes/restarts with backoff.
- ImagePullBackOff — image pull repeatedly failing.

54. Local Hands-On Project

Aapke Node.js background ke liye practical project:

Node.js API
↓
Docker
↓
Kubernetes

Components:
Node API Deployment
Node Service
Ingress
Redis
PostgreSQL
Prometheus
Grafana

- Dockerize Express API.
- Create Deployment with 2 replicas.
- Expose using ClusterIP Service.
- Add Ingress route.
- Add readiness/liveness endpoints.
- Set CPU/memory requests and limits.
- Configure HPA.
- Scale traffic using load generator.
- Observe Pods and p95 latency.
- Kill a Pod and watch replacement.
- Break readiness and observe traffic removal.
- Create rolling deployment v1 → v2.
- Rollback v2 → v1.

55. Interview Q&A;

Question	Short answer
Kubernetes kya?	Container orchestration platform for scheduling, scaling, networking and lifecycle.
Pod?	Smallest deployable Kubernetes unit.
Deployment?	Manages replicated stateless Pods and rollouts.
Service?	Stable network endpoint for Pods.
Ingress?	HTTP/HTTPS routing abstraction into cluster services.
ConfigMap?	Non-secret configuration.
Secret?	Sensitive configuration abstraction; protect access/storage.
Liveness?	Should this container be restarted?
Readiness?	Should this Pod receive traffic?
Startup probe?	Protect slow startup from premature liveness failures.
HPA?	Automatically adjusts replicas based on metrics.
Requests?	Resources scheduler uses for placement/reservation.
Limits?	Upper resource boundaries for container.
StatefulSet?	Stable identity/storage patterns for stateful workloads.
DaemonSet?	Run a Pod on eligible nodes.
Job?	Run finite task to completion.
CronJob?	Schedule Jobs.
RBAC?	Authorization of users/workloads to Kubernetes resources.
NetworkPolicy?	Control Pod network traffic.

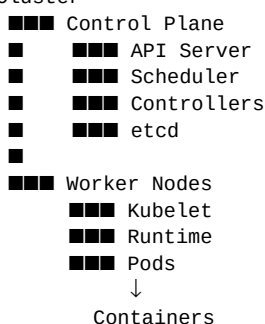
56. Important Interview Scenarios

- A Pod is Running but users get errors — investigate readiness, Service, logs, application and dependencies.
- CPU is 90% but latency is normal — don't blindly scale; understand workload and capacity.
- Pods scaled from 3 to 20 but API still slow — inspect DB/Redis/external dependency bottleneck.
- Deployment v2 causes 5xx — stop/rollback and inspect health, logs and traces.
- Pod keeps restarting — check logs, events, liveness probe, OOM and application crash.
- Traffic should not reach a Pod until DB connection is ready — readiness probe.

57. Final Cheat Sheet

KUBERNETES

Cluster



Application:

Deployment
↓
Pods × N
↓
Service
↓
Ingress/Gateway
↓
Internet

Operations:

- Requests / Limits
- Liveness / Readiness / Startup
- HPA
- Rolling / Canary / Blue-Green
- ConfigMap / Secret
- RBAC
- NetworkPolicy
- PV / PVC
- Observability

Golden interview line:

"Kubernetes lets me declaratively define the desired state of my containerized application. Deployments maintain replicas and rollouts, Services provide stable discovery, Ingress/Gateway handles traffic entry, readiness/liveness probes control health, and HPA handles horizontal scaling. I would also define realistic resource requests/limits and use Prometheus, Grafana and tracing to observe the workload."

58. 80/20 Mental Model

Docker:

"Run my container"

Kubernetes:

"Keep my application running correctly"

Deployment:

"Keep N replicas"

Service:

"Give Pods a stable endpoint"

Ingress:

"Route external HTTP traffic"

Readiness:

"Send traffic only when ready"

Liveness:

"Restart if unhealthy"

HPA:

"Scale replicas"

Requests/Limits:

"Manage resources"

Controllers:

"Continuously reconcile desired vs actual state"